My Project

Generated by Doxygen 1.18.0

1 Hierarchical Index 1

1.1 Class Hierarchy . . . 1

2 Class Index 3

2.1 Class List . . . 3

3 File Index 5

3.1 File List . . . 5

4 Class Documentation 7

4.1 Fabryka Class Reference . . . 7

4.1.1 Detailed Description . . . 7

4.1.2 Member Function Documentation . . . 7

4.1.2.1 utworzWezel() . . . 7

4.2 FabrykaWezlow Class Reference . . . 8

4.2.1 Detailed Description . . . 8

4.2.2 Member Function Documentation . . . 8

4.2.2.1 utworzWezel() . . . 8

4.3 Iterator Class Reference . . . 9

4.3.1 Detailed Description . . . 9

4.3.2 Member Function Documentation . . . 9

4.3.2.1 czyKoniec() . . . 9

4.3.2.2 getAktualny() . . . 9

4.3.2.3 pobierzWartosc() . . . 10

4.4 ListaDwukierunkowa Class Reference . . . 10

4.4.1 Detailed Description . . . 11

4.4.2 Member Function Documentation . . . 11

4.4.2.1 dodajNaKoniec() . . . 11

4.4.2.2 dodajNaPoczatek() . . . 11

4.4.2.3 dodajPodIndeks() . . . 11

4.4.2.4 pobierzRozmiar() . . . 12

4.4.2.5 usunPodIndeks() . . . 12

4.4.2.6 utworzIterator() . . . 12

4.5 Wezel Class Reference . . . 12

4.5.1 Detailed Description . . . 13

4.5.2 Constructor & Destructor Documentation . . . 13

4.5.2.1 Wezel() . . . 13

5 File Documentation 15

5.1 Fabryka.h . . . 15

5.2 Iterator.h . . . 15

5.3 ListaDwukierunkowa.h . . . 16

5.4 Wezel.h . . . 16

Index 17

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Fabryka 7
FabrykaWezlow 8
Iterator 9
ListaDwukierunkowa 10
Wezel 12

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Fabryka
: Abstrakcyjna klasa fabryki do tworzenia obiektów typu Wezel 7

FabrykaWezlow
: Tworzy nowy węzeł z podaną wartością oraz opcjonalnymi wskaźnikami do poprzedniego i następnego węzła 8

Iterator
: Klasa umożliwiająca iterację po elementach listy powiązanej za pomocą wskaźnika na bieżący węzeł 9

ListaDwukierunkowa
: Klasa implementująca dwukierunkową listę liczb całkowitych z obsługą dodawania, usuwania i wyświetlania elementów 10

Wezel
: Reprezentuje węzeł listy dwukierunkowej przechowujący wartość typu int oraz wskaźniki na poprzedni i następny węzeł 12

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Fabryka.h 15
Iterator.h 15
ListaDwukierunkowa.h 16
Wezel.h 16

Chapter 4

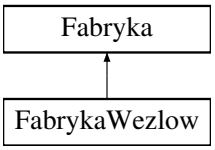
Class Documentation

4.1 Fabryka Class Reference

Abstrakcyjna klasa fabryki do tworzenia obiektów typu Wezel.

`#include <Fabryka.h>`

Inheritance diagram for Fabryka:



Public Member Functions

- virtual Wezel ∗ utworzWezel (int wartosc, Wezel ∗poprzedni=nullptr, Wezel ∗nastepny=nullptr)=0

4.1.1 Detailed Description

Abstrakcyjna klasa fabryki do tworzenia obiektów typu Wezel.

4.1.2 Member Function Documentation

4.1.2.1 utworzWezel()

```
virtual Wezel * Fabryka::utworzWezel (
            int wartosc,
            Wezel * poprzedni = nullptr,
            Wezel * nastepny = nullptr)  [pure virtual]
```

Implemented in FabrykaWezlow.

The documentation for this class was generated from the following file:

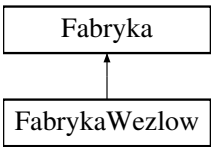
- Fabryka.h

4.2 FabrykaWezlow Class Reference

Tworzy nowy węzeł z podaną wartością oraz opcjonalnymi wskaźnikami do poprzedniego i następnego węzła.

`#include <Fabryka.h>`

Inheritance diagram for FabrykaWezlow:



Public Member Functions

- Wezel * utworzWezel (int wartosc, Wezel *poprzedni=nullptr, Wezel *nastepny=nullptr) override
  Tworzy nowy węzeł z podaną wartością oraz wskaźnikami na poprzedni i następny węzeł.

4.2.1 Detailed Description

Tworzy nowy węzeł z podaną wartością oraz opcjonalnymi wskaźnikami do poprzedniego i następnego węzła.

4.2.2 Member Function Documentation

4.2.2.1 utworzWezel()

```
Wezel * FabrykaWezlow::utworzWezel (
            int wartosc,
            Wezel * poprzedni = nullptr,
            Wezel * nastepny = nullptr)  [override], [virtual]
```

Tworzy nowy węzeł z podaną wartością oraz wskaźnikami na poprzedni i następny węzeł.

Parameters

wartosc	Wartość przechowywana w nowym węźle.
poprzedni	Wskaźnik na poprzedni węzeł (może być nullptr).
nastepny	Wskaźnik na następny węzeł (może być nullptr).

Returns

Wskaźnik do nowo utworzonego węzła.

Implements Fabryka.

The documentation for this class was generated from the following files:

- Fabryka.h
- Fabryka.cpp

4.3 Iterator Class Reference

Klasa umożliwiająca iterację po elementach listy powiązanej za pomocą wskaźnika na bieżący węzeł.

`#include <Iterator.h>`

Public Member Functions

- **Iterator** (Wezel *start)
- bool czyKoniec () const

  Sprawdza, czy iterator osiągnął koniec kolekcji.
- int pobierzWartosc () const

  Zwraca wartość wskazywaną przez iterator.
- void **nastepny** ()

  Przesuwa iterator na następny element w kolekcji, jeśli istnieje.
- void **poprzedni** ()

  Przesuwa iterator na poprzedni element w kolekcji, jeśli to możliwe.
- Wezel * getAktualny () const

  Zwraca wskaźnik do aktualnego węzła iteratora.

4.3.1 Detailed Description

Klasa umożliwiająca iterację po elementach listy powiązanej za pomocą wskaźnika na bieżący węzeł.

4.3.2 Member Function Documentation

4.3.2.1 czyKoniec()

```
bool Iterator::czyKoniec () const
```

Sprawdza, czy iterator osiągnął koniec kolekcji.

Returns

Zwraca true, jeśli iterator wskazuje na koniec (nullptr), w przeciwnym razie false.

4.3.2.2 getAktualny()

```
Wezel * Iterator::getAktualny () const
```

Zwraca wskaźnik do aktualnego węzła iteratora.

Returns

Wskaźnik do aktualnego węzła (Wezel*) lub nullptr, jeśli iterator nie wskazuje na żaden węzeł.

4.3.2.3 pobierzWartosc()

```
int Iterator::pobierzWartosc () const
```

Zwraca wartość wskazywaną przez iterator.

Returns

Wartość elementu, na który wskazuje iterator. Rzuca wyjątek std::out_of_range, jeśli iterator wskazuje na nullptr.

The documentation for this class was generated from the following files:

- Iterator.h
- Iterator.cpp

4.4 ListaDwukierunkowa Class Reference

Klasa implementująca dwukierunkową listę liczb całkowitych z obsługą dodawania, usuwania i wyświetlania elementów.

```
#include <ListaDwukierunkowa.h>
```

Public Member Functions

- **ListaDwukierunkowa** (FabrykaWezlow *fabryka)
- void dodajNaPoczatek (int wartosc)

  Dodaje nowy element na początek listy dwukierunkowej.
- void dodajNaKoniec (int wartosc)

  Dodaje nowy element o podanej wartości na koniec listy dwukierunkowej.
- void dodajPodIndeks (int indeks, int wartosc)

  Dodaje nowy element o podanej wartości na określonym indeksie w liście dwukierunkowej.
- void **usunZPoczatek** ()

  Usuwa pierwszy element z listy dwukierunkowej.
- void **usunZKoniec** ()

  Usuwa ostatni element z listy dwukierunkowej.
- void usunPodIndeks (int indeks)

  Usuwa element z listy dwukierunkowej na podanym indeksie.
- void **wyczysc** ()

  Usuwa wszystkie elementy z listy dwukierunkowej, zwalniając zaalokowaną pamięć.
- void **wyswietl** () const

  Wyświetla zawartość listy dwukierunkowej od początku do końca.
- void **wyswietlOdTylu** () const

  Wyświetla zawartość listy dwukierunkowej od końca do początku.
- Iterator utworzIterator () const

  Tworzy nowy iterator wskazujący na początek listy dwukierunkowej.
- int pobierzRozmiar () const

  Zwraca liczbę elementów w liście dwukierunkowej.

4.4.1 Detailed Description

Klasa implementująca dwukierunkową listę liczb całkowitych z obsługą dodawania, usuwania i wyświetlania elementów.

4.4.2 Member Function Documentation

4.4.2.1 dodajNaKoniec()

```
void ListaDwukierunkowa::dodajNaKoniec (
            int wartosc)
```

Dodaje nowy element o podanej wartości na koniec listy dwukierunkowej.

Parameters

wartosc	Wartość, która zostanie dodana jako nowy element na końcu listy.

4.4.2.2 dodajNaPoczatek()

```
void ListaDwukierunkowa::dodajNaPoczatek (
            int wartosc)
```

Dodaje nowy element na początek listy dwukierunkowej.

Parameters

wartosc	Wartość do dodania na początek listy.

4.4.2.3 dodajPodIndeks()

```
void ListaDwukierunkowa::dodajPodIndeks (
            int indeks,
            int wartosc)
```

Dodaje nowy element o podanej wartości na określonym indeksie w liście dwukierunkowej.

Parameters

indeks	Pozycja, na której ma zostać dodany nowy element. Musi być w zakresie od 0 do rozmiar listy.
wartosc	Wartość nowego elementu do dodania.

4.4.2.4 pobierzRozmiar()

```
int ListaDwukierunkowa::pobierzRozmiar () const
```

Zwraca liczbę elementów w liście dwukierunkowej.

Returns

Aktualny rozmiar (liczbę elementów) listy dwukierunkowej jako wartość typu int.

4.4.2.5 usunPodIndeks()

```
void ListaDwukierunkowa::usunPodIndeks (
            int indeks)
```

Usuwa element z listy dwukierunkowej na podanym indeksie.

Parameters

indeks	Indeks elementu do usunięcia (liczony od zera).

4.4.2.6 utworzIterator()

```
Iterator ListaDwukierunkowa::utworzIterator () const
```

Tworzy nowy iterator wskazujący na początek listy dwukierunkowej.

Returns

Nowy obiekt Iterator ustawiony na pierwszy element listy.

The documentation for this class was generated from the following files:

- ListaDwukierunkowa.h
- ListaDwukierunkowa.cpp

4.5 Wezel Class Reference

Reprezentuje węzeł listy dwukierunkowej przechowujący wartość typu int oraz wskaźniki na poprzedni i następny węzeł.

```
#include <Wezel.h>
```

Public Member Functions

- Wezel (int val, Wezel *pop=nullptr, Wezel *nast=nullptr)

  Konstruktor inicjalizujący obiekt klasy Wezel z podaną wartością oraz wskaźnikami na poprzedni i następny węzeł.

Public Attributes

- int **wartosc**
- Wezel * **nastepny**
- Wezel * **poprzedni**

4.5.1 Detailed Description

Reprezentuje węzeł listy dwukierunkowej przechowujący wartość typu int oraz wskaźniki na poprzedni i następny węzeł.

4.5.2 Constructor & Destructor Documentation

4.5.2.1 Wezel()

```
Wezel::Wezel (
            int val,
            Wezel * pop = nullptr,
            Wezel * nast = nullptr)
```

Konstruktor inicjalizujący obiekt klasy Wezel z podaną wartością oraz wskaźnikami na poprzedni i następny węzeł.

Parameters

val	Wartość przechowywana w węźle.
pop	Wskaźnik na poprzedni węzeł.
nast	Wskaźnik na następny węzeł.

The documentation for this class was generated from the following files:

- Wezel.h
- Wezel.cpp

Chapter 5

File Documentation

5.1 Fabryka.h

```
00001 #ifndef FABRYKA_H
00002 #define FABRYKA_H
00003
00004 #include "Wezel.h"
00005
00009
00010 class Fabryka {
00011 public:
00012     virtual ~Fabryka() = default;
00013     virtual Wezel* utworzWezel(int wartosc, Wezel* poprzedni = nullptr, Wezel* nastepny = nullptr) =
      0;
00014 };
00015
00019
00020 class FabrykaWezlow : public Fabryka {
00021 public:
00022     Wezel* utworzWezel(int wartosc, Wezel* poprzedni = nullptr, Wezel* nastepny = nullptr) override;
00023 };
00024
00025 #endif
```

5.2 Iterator.h

```
00001 #ifndef ITERATOR_H
00002 #define ITERATOR_H
00003
00004 #include "Wezel.h"
00005
00009
00010 class Iterator {
00011 private:
00012     Wezel* aktualny;
00013
00014 public:
00015     Iterator(Wezel* start);
00016
00017     bool czyKoniec() const;
00018
00019     int pobierzWartosc() const;
00020
00021     void nastepny();
00022
00023     void poprzedni();
00024
00025     Wezel* getAktualny() const;
00026 };
00027
00028 #endif
```

5.3 ListaDwukierunkowa.h

```
00001 #ifndef LISTA_DWUKIERUNKOWA_H
00002 #define LISTA_DWUKIERUNKOWA_H
00003
00004 #include "Wezel.h"
00005 #include "fabryka.h"
00006 #include "Iterator.h"
00007
00011
00012 class ListaDwukierunkowa {
00013 private:
00014     Wezel* poczatek;
00015     Wezel* koniec;
00016     int rozmiar;
00017     FabrykaWezlow* fabryka;
00018
00019 public:
00020     ListaDwukierunkowa(FabrykaWezlow* fabryka);
00021     ~ListaDwukierunkowa();
00022
00023
00024     void dodajNaPoczatek(int wartosc);
00025     void dodajNaKoniec(int wartosc);
00026     void dodajPodIndeks(int indeks, int wartosc);
00027
00028
00029     void usunZPoczatek();
00030     void usunZKoniec();
00031     void usunPodIndeks(int indeks);
00032
00033
00034     void wyczysc();
00035
00036     void wyswietl() const;
00037     void wyswietlOdTylu() const;
00038
00039
00040     Iterator utworzIterator() const;
00041
00042     int pobierzRozmiar() const;
00043 };
00044
00045 #endif
```

5.4 Wezel.h

```
00001 #ifndef WEZEL_H
00002 #define WEZEL_H
00003
00007
00008 class Wezel {
00009 public:
00010     int wartosc;
00011     Wezel* nastepny;
00012     Wezel* poprzedni;
00013
00014     Wezel(int val, Wezel* pop = nullptr, Wezel* nast = nullptr);
00015 };
00016
00017 #endif
```

Index

czyKoniec
    Iterator, 9

dodajNaKoniec
    ListaDwukierunkowa, 11
dodajNaPoczatek
    ListaDwukierunkowa, 11
dodajPodIndeks
    ListaDwukierunkowa, 11

Fabryka, 7
    utworzWezel, 7
FabrykaWezlow, 8
    utworzWezel, 8

getAktualny
    Iterator, 9

Iterator, 9
    czyKoniec, 9
    getAktualny, 9
    pobierzWartosc, 9

ListaDwukierunkowa, 10
    dodajNaKoniec, 11
    dodajNaPoczatek, 11
    dodajPodIndeks, 11
    pobierzRozmiar, 11
    usunPodIndeks, 12
    utworzIterator, 12

pobierzRozmiar
    ListaDwukierunkowa, 11
pobierzWartosc
    Iterator, 9

usunPodIndeks
    ListaDwukierunkowa, 12
utworzIterator
    ListaDwukierunkowa, 12
utworzWezel
    Fabryka, 7
    FabrykaWezlow, 8

Wezel, 12
    Wezel, 13